

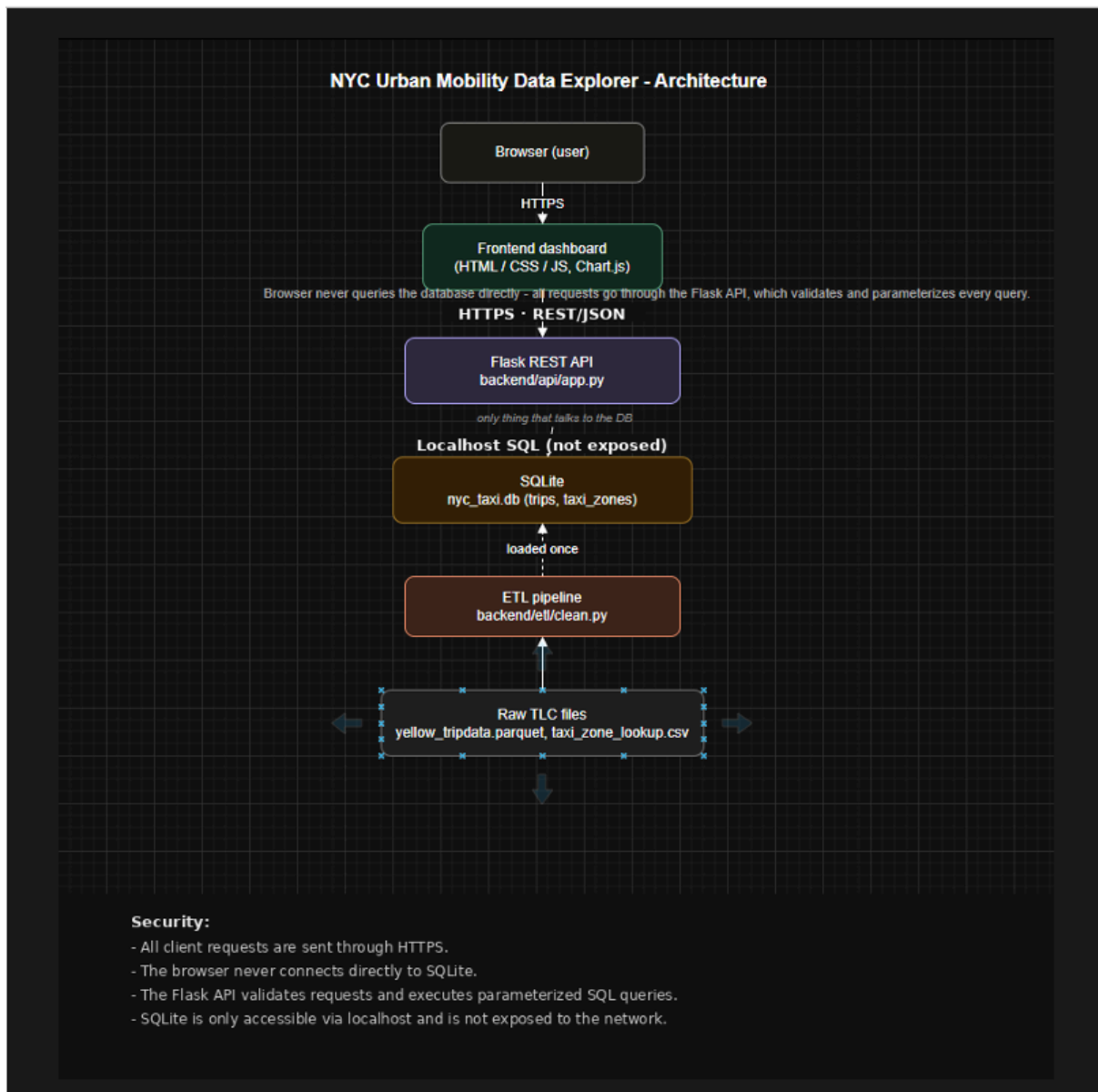
NYC Urban Mobility Data Explorer

Problem:

Raw NYC TLC taxi data comes as disconnected files: trip records, a zone lookup table, and zone boundaries with no way to answer basic questions like which boroughs earn the most, when demand peaks, or how airport trips differ from local ones. This project cleans and loads a full month of trip data (7.67 million raw rows) into a relational database, serves it through a REST API, and exposes it through a dashboard that lets a user explore those patterns without writing SQL.

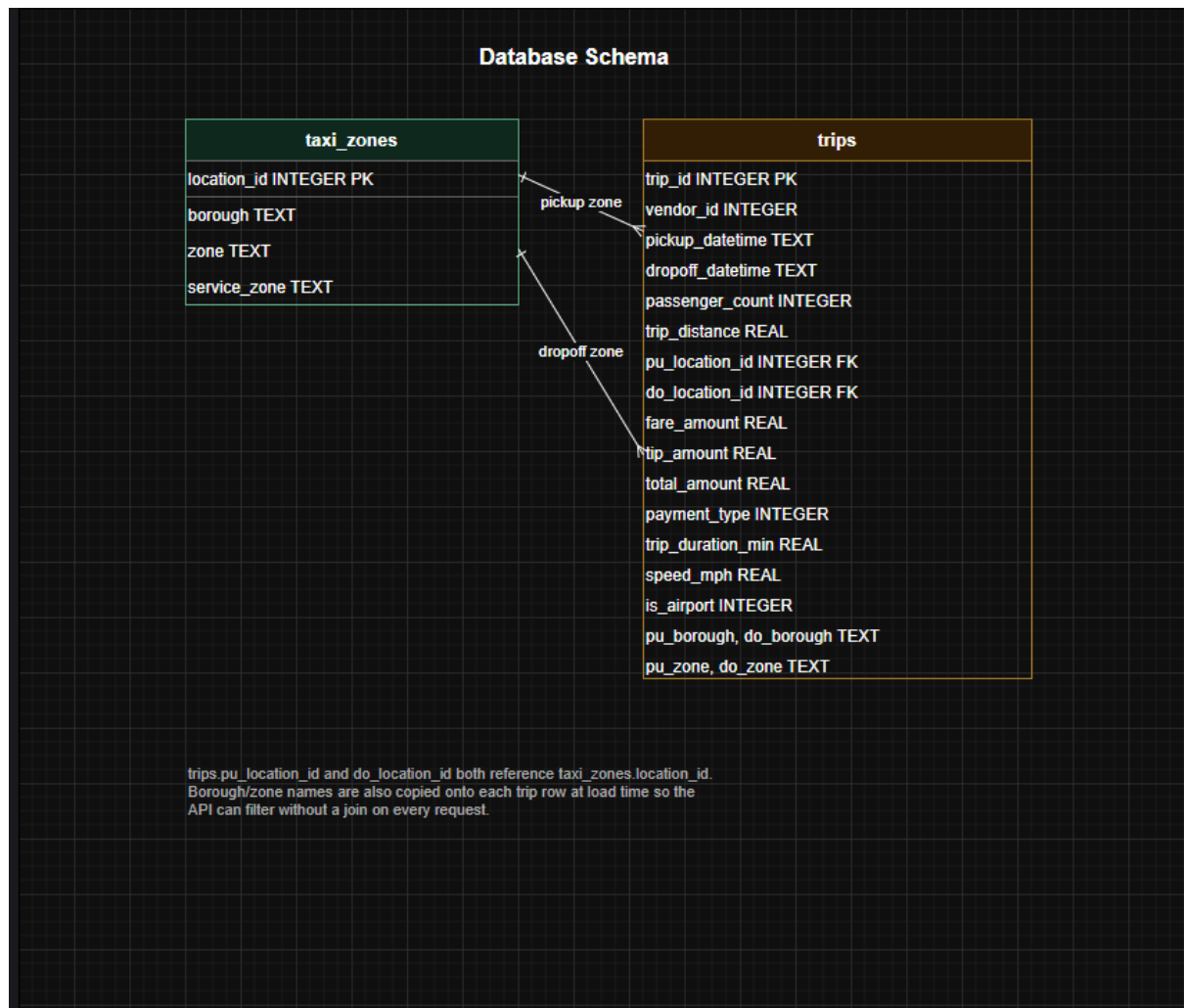
Architecture:

The browser only ever talks to the static frontend and to the Flask API over HTTPS. The Flask API is the only component allowed to touch the database: it validates every request and uses parameterized SQL over a localhost-only connection, so the database is never exposed to the network directly. The ETL pipeline is a separate offline job: it runs once to load and clean the raw data into SQLite before the API or frontend ever serve a request. Keeping these three concerns (data loading, data serving, data display) separate lets the three of us build them in parallel. Communication between the client and server occurs over HTTPS using REST/JSON. The SQLite database is never directly accessible to clients; all database operations are performed through the Flask API using parameterized SQL queries over a localhost-only connection.



Database Design:

The schema is one denormalized trips table plus a taxi_zones reference table, joined once at load time rather than on every query. Borough and zone names are copied onto each trip row during the ETL so the API can filter and group by location without a runtime join. Five indexes (pickup_date, pu_borough, hour, payment_type, is_airport) cover the dashboard's actual filter and group-by patterns rather than indexing every column .



Algorithms:

Three algorithms were implemented by hand in backend/etl/algorithms.py instead of using Python's built-in sort, heapq, or Counter, and are exposed live through /api/dsa-benchmark:

Algorithm	Used for
QuickSort	Sorting trips or zones by a numeric key
Min-heap	Finding the top-10 busiest pickup zones without sorting all- 260 zones
HashMap (chaining)	Counting trips per borough

Reflection:

The ability to understand how the schema of the trips table and stats.json looks like beforehand helped us to work in parallel since Olivier was working on ETL and the database, Gabriel on the API, and James on the dashboard as well as frontend, testing and documenting the code. One of the biggest lessons that we learned during this assignment was that checking whether the record is bad or not should be based on the data itself and not on any assumptions before. Instead of skipping bad records silently, logging them all let us check whether the data cleansing is performed correctly or not. If we had some extra time, then expanding the time series from one month to twelve would be an obvious thing to do, which would mean moving to PostgreSQL from SQLite.